EnvBoard

Product & Technical Requirements

Product ask

1) Ability for an admin to define and maintain environments.

a) Define a new environment (name, description, console URL).
b) Mark active/inactive.
c) Edit environment details.
d) View which environments are currently held and by whom.
e) Force-reclaim any active hold, with a mandatory reason.

2) Ability for a developer to reserve an environment.

a) Provide purpose and time required for testing.
b) Add additional minutes if testing takes longer than expected.
c) Release the environment early once done.
d) See a message naming the holder if the environment is already taken.
e) Hold at most 2 environments at a time.

3) Ability for anyone to view live board state.

a) See every environment's status: available, in use, or unavailable.
b) See holder, purpose, and time remaining for environments in use.
c) See changes reflected across all open boards within seconds, no refresh.

4) Ability for holds to expire automatically.

a) Hold ends on its own once time runs out — no manual cleanup.
b) Environment shows available immediately on expiry, independent of any background job.
c) No grace period and no auto-renewal — must extend before expiry.

5) Ability for anyone to view history and audit trail.

a) See a per-environment log: claimed, extended, released, expired, reclaimed.
b) Each entry shows who, what, when, and why (where applicable).
c) History is append-only — nothing can be edited or deleted.

6) Ability to identify and authorize users.

a) Every action tied to an identified person.
b) Two roles: member (claim/extend/release own holds) and admin (all of the above, plus reclaim and manage environments).

Tech requirements

1) Stack: Go service, MySQL as the single source of truth.

a) No separate cache or broker in v1.

2) MySQL used for locking and synchronization.

a) Simultaneous claims on the same environment must resolve to exactly one winner.
b) Expiry must be correct at read time, not dependent on a background job.

3) Service must be horizontally scalable.

a) Stateless — no pod-local session or hold state.

b) Any pod can serve any request.

4) Rate limiting.

a) Per-user limits on write actions (claim/extend/release).

5) User and admin management.

a) No self-service signup.

b) Every action attributed to an identified user.

c) Admin actions enforced server-side, not just hidden in the UI.

Optional

1) Ability for admins to see environment utilization as a heat map.

a) Usage per environment over time, to decide whether to add or remove environments.

2) Ability for developers to register intent to test when no environment is free.

a) Shows demand waiting on availability, separate from raw environment count — surfaces a time bottleneck vs. a capacity bottleneck.

Note

Use of AI to write this project should be very limited. Most of the code should be written by hand, so the exercise actually builds understanding rather than being generated around.